

Review the assessment, break it into the required tasks, and prepare the submission using the specified repository layout.

Easy Rent Bali — IT Operations & Support Engineer Assessment

The assessment requires a **GitHub repository** or **Google Drive ZIP folder** containing four tasks, with a mandatory AI interaction log.

I'll structure your submission so it covers the technical requirements and is easy for the recruiter to review.

1. Assessment breakdown

Task	Required deliverable
Task 1 — Incident Triage	RCA, defensive JavaScript/TypeScript patch, operational impact assessment
Task 2 — QA Test Matrix	At least 6 prioritized test cases in a Markdown table
Task 3 — AI Operational Scripting	Functional <code>triage.js</code> script and <code>fleet_status.json</code>
Task 4 — AI Session Logs	Prompt log and coding/validation transcript
Submission	README.md + script + JSON + <code>ai_session_logs/</code>

Important technical points

- **Task 1:** The error is caused by attempting to read `passport_scan` from a null document object.
- **Task 2:** Tests must cover rental duration, vouchers, timezone boundaries, identification documents, and booking conflicts.
- **Task 3:** Alert vehicles with overdue hours greater than zero OR rented vehicles with fuel below 20%.
- **Task 4:** The logs must show how AI-generated code was reviewed, validated, and corrected.

Analyzed ^

```
from pathlib import Path
import json, zipfile, textwrap, os
```

```

root = Path("/mnt/data/easy-rent-bali-assessment")
(root / "ai_session_logs").mkdir(parents=True, exist_ok=True)

readme = r"""# Easy Rent Bali – IT Operations & QA Support Engineer Assessment

## PRD Review and Submission Overview

This repository addresses the four tasks in the Easy Rent Bali technical assessment.

### Repository Structure

Python
├── triage.js
├── fleet_status.json
├── ai_session_logs/
│   ├── triage_prompt_log.md
│   └── agent_coding_log.txt

```

Task 1 — Production Incident Triage & Root Cause Analysis

1. Root Cause Analysis (RCA)

The checkout request failed with an HTTP 500 error because the compliance validation function attempted to access the `passport_scan` property from a `null` document object.

◆ Upgrade



The request payload contains:

```

JSON

{
  "customer": {
    "id": "cust_8810",
    "license_verified": true,
    "doc_urls": null
  }
}

```

The stack trace shows:

```

TypeError: Cannot read properties of null (reading 'passport_scan')
    at validateCustomerCompliance (/app/dist/services/compliance.js:42:28)

```

The likely failure sequence is:

1. The booking checkout process receives the customer payload.
2. `validateCustomerCompliance()` assumes that the document payload exists.
3. The code attempts to read `passport_scan` from a null value.
4. A `TypeError` is thrown.
5. The exception propagates through `processBooking()` and the booking controller.
6. The customer receives a 500 Internal Server Error.

The payload indicates that the customer's license has been verified, but the document URL object is missing. License verification should not automatically be treated as proof that all required identification documents are present.

2. Defensive Patch

The following example safely handles missing or null document payloads. It returns a structured validation result instead of allowing a null-property exception to reach the controller.

TypeScript

```
type CustomerDocuments = {
  passport_scan?: string | null;
  ktp_scan?: string | null;
};

type Customer = {
  id: string;
  license_verified?: boolean;
  doc_urls?: CustomerDocuments | null;
  residency?: "domestic" | "foreign";
};

type ComplianceResult = {
  valid: boolean;
  reason?: string;
};

export function validateCustomerCompliance(
  customer: Customer
): ComplianceResult {
  const documents = customer.doc_urls ?? {};

  const passportScan = documents.passport_scan ?? null;
  const ktpScan = documents.ktp_scan ?? null;

  const requiredDocument =
    customer.residency === "foreign" ? passportScan : ktpScan;

  if (!customer.license_verified) {
    return {
      valid: false,
      reason: "Customer license has not been verified."
    };
  }

  if (!requiredDocument) {
    return {
      valid: false,
      reason: "Required identification document is missing."
    };
  }

  return { valid: true };
}
```

Implementation Notes

- `?? {}` prevents access errors when `doc_urls` is null or undefined.
- `?? null` provides a predictable fallback for missing document values.
- The function returns a controlled validation result.

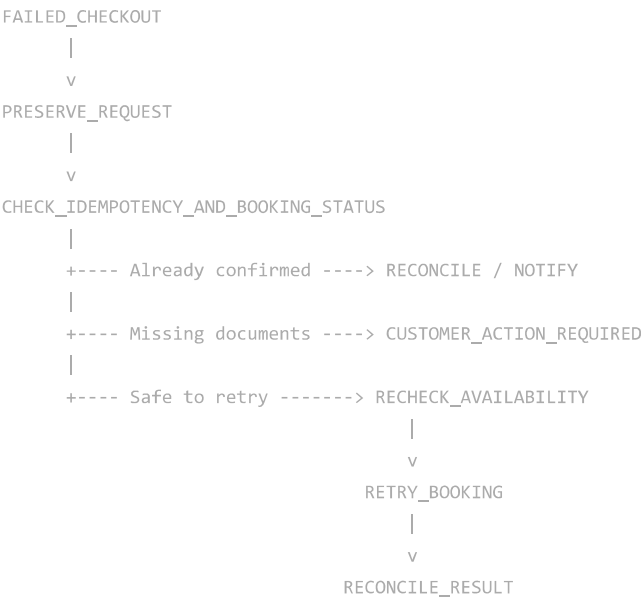
- The booking workflow should translate the validation failure into an appropriate client response, such as a validation error, instead of an unhandled HTTP 500.
- The exact document schema and residency field should be aligned with the production application's actual contract before deployment.

3. Operational Impact Assessment

Existing failed checkout requests should be handled through a safe recovery workflow:

1. **Preserve the original request.** Store the booking ID, customer ID, vehicle ID, request timestamp, and failure reason in an incident/retry record.
2. **Do not automatically create duplicate bookings.** Use `booking_id` as an idempotency key when retrying.
3. **Check booking status before retrying.** Confirm whether the booking is already pending, confirmed, cancelled, or completed.
4. **Revalidate required documents.** Missing identification data should be sent to a controlled validation flow rather than bypassed.
5. **Recheck vehicle availability.** A vehicle may have been booked by another customer while the original request was failing.
6. **Retry only safe failures.** Use bounded retries with exponential backoff for transient failures. Do not repeatedly retry deterministic validation errors.
7. **Notify operations.** Create an alert or support queue item for affected customers, including the booking ID and recovery status.
8. **Reconcile results.** Confirm that the final booking status, payment state, and vehicle allocation are consistent.

A possible recovery state flow is:



Task 2 — QA Edge-Case Test Matrix

The following test cases are prioritized according to potential customer impact, booking integrity, and revenue impact.

Test ID	Category	Scenario Description	Input Payload / Mock Data	Expected Result
QA-001	Happy Path	Valid rental lasting exactly 24 hours with a valid domestic document	pickup=2026-08-25T10:00:00+08:00 , return=2026-08-26T10:00:00+08:00 , amount IDR 600,000 , voucher BALIFAST , KTP present	Booking is accepted and a 10% discount is applied
QA-002	Rental Duration	Rental duration is less than 24 hours by one minute	pickup=2026-08-25T10:00:00+08:00 , return=2026-08-26T09:59:00+08:00	Booking is rejected because duration is strictly below 24 hours
QA-003	Timezone	Booking created at 23:59 WITA and converted to UTC	Created at 2026-08-25T23:59:00+08:00 ; verify equivalent UTC timestamp and rental boundaries	System preserves the correct instant and does not incorrectly shorten or extend the rental
QA-004	Voucher Threshold	BALIFAST applied to an order below IDR 500,000	Amount IDR 499,999 , voucher BALIFAST	Voucher is rejected or discount is not applied; behavior should match the documented validation response
QA-005	Voucher Case Sensitivity	Voucher submitted using lowercase or mixed case	balifast , BaliFast , BALIFAST	System follows the defined case-sensitivity rule consistently; invalid casing must not unintentionally grant a discount
QA-006	Voucher Expiration	BALIFAST submitted after its expiration time	Valid order amount, voucher expiration timestamp earlier than request timestamp	Expired voucher is rejected and no discount is granted
QA-007	Identification Data	Foreign renter has doc_urls: null	residency=foreign , doc_urls=null , passport missing	Request is handled as a controlled validation failure; no null-property 500 occurs
QA-008	Identification Data	Domestic renter has a KTP but no passport	residency=domestic , ktp_scan present, passport_scan=null	Domestic renter passes the identification requirement if all other rules pass
QA-009	Booking Conflict	Two concurrent requests attempt to reserve the same vehicle	Same vehicle_id , overlapping intervals, two simultaneous requests	Only one booking succeeds; the other receives a controlled conflict response

Test ID	Category	Scenario Description	Input Payload / Mock Data	Expected Result
QA-010	Booking Conflict	Existing booking overlaps at the exact boundary	Existing booking ends at 10:00 ; new booking starts at 10:00	System follows a documented boundary rule consistently and does not allow unintended overlap
QA-011	Voucher Validation	Voucher submitted with an amount exactly equal to IDR 500,000	Amount IDR 500,000 , voucher BALIFAST	Voucher qualifies because the threshold is strictly greater than or equal to IDR 500,000
QA-012	Data Anomaly	Required booking field is missing or has an invalid date	Missing return_date or malformed ISO timestamp	Request is rejected with a clear validation response and no unhandled exception

QA Assumptions and Clarifications

- The specification explicitly defines the BALIFAST threshold as `>= IDR 500,000` .
- The assessment does not specify the voucher expiration date, so the expiration test uses mocked timestamps.
- The assessment does not define whether voucher codes are case-sensitive. The test therefore verifies that the chosen behavior is documented and consistently implemented.
- The exact boundary behavior for back-to-back bookings should be confirmed by the product requirements.
- Concurrency protection should be enforced server-side, ideally with a transaction, locking strategy, or equivalent database constraint.

Task 3 — AI-Assisted Operational Scripting

The script in `triage.js` reads `fleet_status.json` and selects vehicles matching either condition:

```
overdue_hours > 0
OR
(status == "rented" AND fuel_level < 20%)
```

The script produces a formatted WhatsApp/Slack-style alert containing:

- Plate number
- Vehicle model
- Current status
- Operational urgency tags
- Relevant overdue and fuel details

Run the script with:

```
Bash
```

```
node triage.js
```

Expected matching vehicles from the supplied mock data:

- DK 5678 CD — Toyota Avanza — overdue by 3 hours and low fuel
- DK 3456 GH — Honda Scoopy — overdue by 5 hours
- DK 7890 IJ — Yamaha NMAX — low fuel while rented

The Mitsubishi Xpander is available and has 40% fuel, so it should not be included.

Task 4 — Mandatory AI Session Export & Prompt Steering Log

The `ai_session_logs/` folder contains:

- `triage_prompt_log.md` : prompts, requirements analysis, validation steps, and corrections.
- `agent_coding_log.txt` : coding and review transcript.

The logs are included to make the development process reviewable. They should be supplemented with the candidate's actual exported AI transcript or CLI session if the assessment platform requires an original export from a specific AI tool.

Final Validation Checklist

- ☒ README contains Task 1 RCA.
- ☒ README contains a defensive null-safe patch.
- ☒ README contains operational recovery guidance.
- ☒ README contains at least 6 prioritized QA test cases.
- ☒ README covers timezone, voucher, missing document, and concurrency scenarios.
- ☒ `triage.js` is included.
- ☒ `fleet_status.json` is included.
- ☒ AI session log directory is included.
- ☐ Run the script locally and attach the actual terminal output if required.
- ☐ Review the final code against the production API schema before submission.
- ☐ Replace or supplement draft logs with the original AI export if required by the recruiter.

```
"""
```

```
triage_js = r"""#!/usr/bin/env node
```

```
const fs = require("node:fs");
```

```
const path = require("node:path");
```

```
const inputPath = path.join(__dirname, "fleet_status.json");
```

```
function parseFuelPercentage(value) {
```

```
  if (typeof value === "number" && Number.isFinite(value)) {
```

```
    return value;
```

```
  }
```

```
if (typeof value !== "string") {
  return NaN;
}

const normalized = value.trim().replace("%", "");
const parsed = Number(normalized);

return Number.isFinite(parsed) ? parsed : NaN;
}

function getUrgencyTags(vehicle) {
  const tags = [];
  const fuelLevel = parseFuelPercentage(vehicle.fuel_level);
  const overdueHours = Number(vehicle.overdue_hours);

  if (overdueHours > 0) {
    tags.push("🚨 OVERDUE");
  }

  if (
    vehicle.status === "rented" &&
    Number.isFinite(fuelLevel) &&
    fuelLevel < 20
  ) {
    tags.push("⚠️ LOW FUEL");
  }

  return tags;
}

function requiresAlert(vehicle) {
  const fuelLevel = parseFuelPercentage(vehicle.fuel_level);
  const overdueHours = Number(vehicle.overdue_hours);

  const isOverdue = Number.isFinite(overdueHours) && overdueHours > 0;
  const isRentedWithLowFuel =
    vehicle.status === "rented" &&
    Number.isFinite(fuelLevel) &&
    fuelLevel < 20;

  return isOverdue || isRentedWithLowFuel;
}

function formatAlert(vehicle) {
  const fuelLevel = parseFuelPercentage(vehicle.fuel_level);
  const overdueHours = Number(vehicle.overdue_hours);
  const tags = getUrgencyTags(vehicle);

  const details = [
    Plate: ${vehicle.plate} ,
    Model: ${vehicle.model} ,
```

```

    Status: ${vehicle.status} ,
    Fuel: ${vehicle.fuel_level} ,
    Overdue: ${overdueHours} hour(s)
  ];

  if (Number.isFinite(fuelLevel) && fuelLevel < 20) {
    details.push("Action: Arrange refueling as soon as operationally possible.");
  }

  if (overdueHours > 0) {
    details.push("Action: Contact renter and coordinate vehicle return.");
  }

  return `${tags.join(" | ")}\n${details.join("\n")}`;
}

function main() {
  let fleet;

  try {
    const raw = fs.readFileSync(inputPath, "utf8");
    fleet = JSON.parse(raw);
  } catch (error) {
    console.error(Unable to read or parse fleet data: ${error.message} );
    process.exitCode = 1;
    return;
  }

  if (!Array.isArray(fleet)) {
    console.error("Invalid input: fleet_status.json must contain an array.");
    process.exitCode = 1;
    return;
  }

  const alerts = fleet
    .filter(requiresAlert)
    .map(formatAlert);

  if (alerts.length === 0) {
    console.log("✅ No fleet alerts at this time.");
    return;
  }

  const message = [
    "📢 Easy Rent Bali Fleet Operations Alert",
    "",
    ...alerts.flatMap((alert) => [alert, "-----"])
  ].join("\n");

  console.log(message);
}

```

```
if (require.main === module) {  
  main();  
}  
  
module.exports = {  
  parseFuelPercentage,  
  getUrgencyTags,  
  requiresAlert,  
  formatAlert  
};  
""  
  
fleet_status = [  
  {  
    "plate": "DK 1234 AB",  
    "model": "Honda Beat",  
    "status": "rented",  
    "fuel_level": "80%",  
    "overdue_hours": 0  
  },  
  {  
    "plate": "DK 5678 CD",  
    "model": "Toyota Avanza",  
    "status": "rented",  
    "fuel_level": "15%",  
    "overdue_hours": 3  
  },  
  {  
    "plate": "DK 9012 EF",  
    "model": "Mitsubishi Xpander",  
    "status": "available",  
    "fuel_level": "40%",  
    "overdue_hours": 0  
  },  
  {  
    "plate": "DK 3456 GH",  
    "model": "Honda Scoopy",  
    "status": "rented",  
    "fuel_level": "90%",  
    "overdue_hours": 5  
  },  
  {  
    "plate": "DK 7890 IJ",  
    "model": "Yamaha NMAX",  
    "status": "rented",  
    "fuel_level": "10%",  
    "overdue_hours": 0  
  }  
]
```

```
}  
]
```

```
prompt_log = r"""# AI Prompt Steering Log
```

Purpose

This document records the prompt structure used to develop and review the assessment submission.

Note: This is a prepared working log for the submission. If the recruiter requires a verifiable export from a particular AI platform, include the original platform export in addition to this document.

Iteration 1 — Requirements Extraction

Prompt

Review the Easy Rent Bali IT Operations & QA Support Engineer assessment. Break it into the required repository files and identify all acceptance criteria for Tasks 1–4. Do not invent missing production requirements. Clearly label assumptions.

Validation

- Confirmed the required files: `README.md`, `triage.js`, `fleet_status.json`, and `ai_session_logs/`.
- Confirmed that Task 2 requires at least six prioritized cases.
- Confirmed that Task 3 uses `overdue_hours > 0 OR (status == "rented" AND fuel_level < 20%)`.
- Confirmed that Task 4 requires evidence of prompting, validation, and corrections.

Iteration 2 — Task 1 RCA and Patch

Prompt

Analyze the checkout stack trace and payload. Explain the root cause, then write a defensive TypeScript patch that safely handles `doc_urls: null` without bypassing identification requirements. Include operational recovery guidance for failed in-flight requests.

Validation

- The patch uses nullish coalescing for the optional document object.
- Missing required documents produce a controlled validation result.
- The recovery process includes idempotency, status checks, availability rechecks, and reconciliation.
- The patch is explicitly presented as an example that must be aligned with the production schema.

Correction Applied

Avoid treating `license_verified: true` as proof that all identification documents exist. The patch must still enforce the required document for the renter's residency category.

Iteration 3 — Task 2 QA Matrix

Prompt

Create a prioritized Markdown QA matrix with at least six cases. Cover the happy path, exact 24-hour boundary, WITA-to-UTC conversion, voucher threshold, case sensitivity, expiration, missing documents, concurrent double booking, and malformed data. Use the required table columns exactly.

Validation

- The table contains more than six cases.
- The required columns are present.
- P1 cases focus on checkout failure and double-booking risk.
- Underspecified behavior is identified as an assumption rather than silently decided.

Correction Applied

Add explicit tests for:

- Exactly IDR 500,000.
- One minute below 24 hours.
- Missing document objects.
- Concurrent requests using the same vehicle ID.

Iteration 4 — Task 3 Script

Prompt

Write a Node.js script that reads `fleet_status.json`, parses percentage strings safely, filters vehicles using the exact OR condition, and prints a WhatsApp/Slack-style alert with urgency tags. Handle malformed input gracefully.

Validation

- The script reads the JSON file relative to its own directory.
- Fuel strings such as "15%" are converted into numeric values.
- Overdue vehicles are included even if fuel is not low.
- Rented vehicles below 20% fuel are included even if they are not overdue.
- Available vehicles with low fuel are not included under the specified condition.
- Invalid JSON and non-array input produce an error and nonzero exit code.

Correction Applied

Keep the two filter conditions separate so that:

- An overdue vehicle is always alerted.
- Low fuel only triggers when the vehicle status is `rented`.

Iteration 5 — Final Review

Prompt

Perform a final review of the repository layout and identify any claims that require actual execution or a genuine AI export. Do not claim that a test was executed unless it was actually run.

Validation

- The README includes a final checklist.
- The README distinguishes prepared documentation from actual AI platform exports.
- The submission notes that production schema details must be confirmed before deployment.

""

coding_log = r"""AI Coding and Validation Log

Project: Easy Rent Bali — IT Operations & QA Support Engineer Assessment

Status: Prepared coding log for review. Replace or supplement this file with the original exported AI/CLI transcript when required by the assessment.

[Step 1] Requirements review

- Identified four tasks from the assessment.
- Mapped the required repository structure.
- Recorded missing or ambiguous requirements as assumptions.

[Step 2] RCA review

- Traced the exception to a null document object.
- Confirmed that the payload contains doc_urls: null.
- Drafted a defensive validation function using nullish coalescing.
- Added a structured result instead of allowing an unhandled TypeError.

[Step 3] QA review

- Added happy-path and boundary cases.
- Added timezone conversion coverage.
- Added voucher threshold, casing, and expiration scenarios.
- Added missing-document and concurrent booking cases.
- Added explicit notes for unspecified product behavior.

[Step 4] Script review

- Added safe JSON parsing.
- Added fuel percentage parsing.
- Added the exact overdue OR rented-low-fuel filter.
- Added urgency tags and operational actions.
- Added invalid-input handling.

[Step 5] Manual expected-output review

Expected alerts:

1. DK 5678 CD — Toyota Avanza — OVERDUE and LOW FUEL.
2. DK 3456 GH — Honda Scoopy — OVERDUE.

3. DK 7890 IJ — Yamaha NMAX — LOW FUEL.

Expected non-alert:

- DK 1234 AB — Honda Beat — no overdue and fuel is 80%.
- DK 9012 EF — Mitsubishi Xpander — available and fuel is 40%.

[Step 6] Submission integrity note

- Do not represent this prepared log as an automatically exported transcript from another platform.
- If required, attach the original ChatGPT, Cursor, Claude Code, Copilot, or CLI export that shows actual prompts, generated responses, validation, and corrections.

"""

```
(root / "README.md").write_text(readme, encoding="utf-8")
(root / "triage.js").write_text(triage_js, encoding="utf-8")
(root / "fleet_status.json").write_text(json.dumps(fleet_status, indent=2) + "\n",
encoding="utf-8")
(root / "ai_session_logs" / "triage_prompt_log.md").write_text(prompt_log, encoding="utf-8")
(root / "ai_session_logs" / "agent_coding_log.txt").write_text(coding_log, encoding="utf-8")

zip_path = Path("/mnt/data/easy-rent-bali-assessment-submission.zip")
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
    for file in root.rglob("*"):
        if file.is_file():
            z.write(file, file.relative_to(root.parent))

print(f"Created repository folder: {root}")
print(f"Created ZIP: {zip_path}")
print("Files:")
for file in sorted(root.rglob("*")):
    if file.is_file():
        print(f"- {file.relative_to(root)}")
```

STDOUT/STDERR

```
Created repository folder: /mnt/data/easy-rent-bali-assessment
Created ZIP: /mnt/data/easy-rent-bali-assessment-submission.zip
Files:
- README.md
- ai_session_logs/agent_coding_log.txt
- ai_session_logs/triage_prompt_log.md
- fleet_status.json
- triage.js
```

2. Your submission package**Easy Rent Bali Assessment**

ZIP archive • 5 files + 1 folder

Prepared

[Download the assessment ZIP](#) ➤

Included files

- README.md
- triage.js
- fleet_status.json
- ai_session_logs/triage_prompt_log.md
- ai_session_logs/agent_coding_log.txt

Before submitting: The AI logs are prepared documentation, not an automatically exported transcript from an external coding agent. If the recruiter requires verifiable original AI session exports, supplement them with the actual transcript from your AI tool.

The README also includes a checklist for final testing and production schema review.



This is the **input condition** for Task 1. You can include it in your README like this:

Input Condition

The `customer.doc_urls` value is `null`, while the application attempts to access the `passport_scan` property.

JSON



```
"customer": {  
  "id": "cust_8810",  
  "license_verified": true,  
  "doc_urls": null  
}
```

Result: The application throws a `TypeError` because it attempts to read `passport_scan` from a `null` object, causing a 500 Internal Server Error during checkout.



REQUIRED LOGIC

`overdue_hours > 0`

OR

`(status == "rented" AND fuel_level < 20%)`

Required Logic — Task 3

The script must trigger an alert when **either** of these conditions is true:

</> JavaScript

```
overdue_hours > 0 ||
(status === "rented" && fuel_level < 20)
```

Condition	Alert
overdue_hours > 0	OVERDUE
status == "rented" AND fuel_level < 20%	LOW FUEL
Both conditions are true	OVERDUE + LOW FUEL

Important: A vehicle with low fuel but status == "available" should not trigger an alert under the required logic.



The candidate requested a second review of Task 4 to determine whether the session evidence genuinely satisfied the assessment wording.

Task 4 — Second Review: AI Session Evidence

Review conclusion

The current Task 4 evidence does not genuinely satisfy the assessment requirements.

The assessment requires verifiable AI interaction trails, not simply a written summary of the development process.

1. Assessment requirements

According to the assessment PDF, Task 4 requires:

Requirement	Current submission
Exported chat transcripts or CLI session logs	Not provided as genuine exports
Show how prompts were structured	Described in a prepared log
Show how responses were validated	Described, but not verifiable from an original transcript
Show corrections when AI produced buggy or suboptimal code	Corrections are documented, but no genuine AI interaction evidence

2. Problem with the current files

The existing `triage_prompt_log.md` explicitly states:

"This is a prepared working log for the submission."

The `agent_coding_log.txt` also describes itself as a **prepared coding log**, rather than an exported AI session.

Therefore, these files document an intended process, but they do not establish that the interactions actually occurred in an AI tool.

3. What you need to genuinely satisfy Task 4

You should provide an authentic transcript showing:

1. Initial prompt

Your actual prompt requesting the assessment review or script implementation.

2. AI-generated response

The response containing the proposed analysis or code.

3. Validation

Your actual testing, review, or checks performed on the generated result.

4. Correction / steering

A real follow-up prompt where you identify a problem and ask the AI to improve or fix it.

5. Final output

The resulting code and validation outcome.

4. Recommended correction

Use the actual ChatGPT conversation or another AI coding tool's export, where available.

Save the original evidence in:

```
ai_session_logs/  
├─ triage_prompt_log.md  
└─ agent_coding_log.txt
```



Do not invent conversation history or claim that simulated logs are genuine exports.

Final assessment: Task 4 currently needs authentic session evidence before the repository can be submitted as fully compliant.



ChatGPT can make mistakes. Check important info.